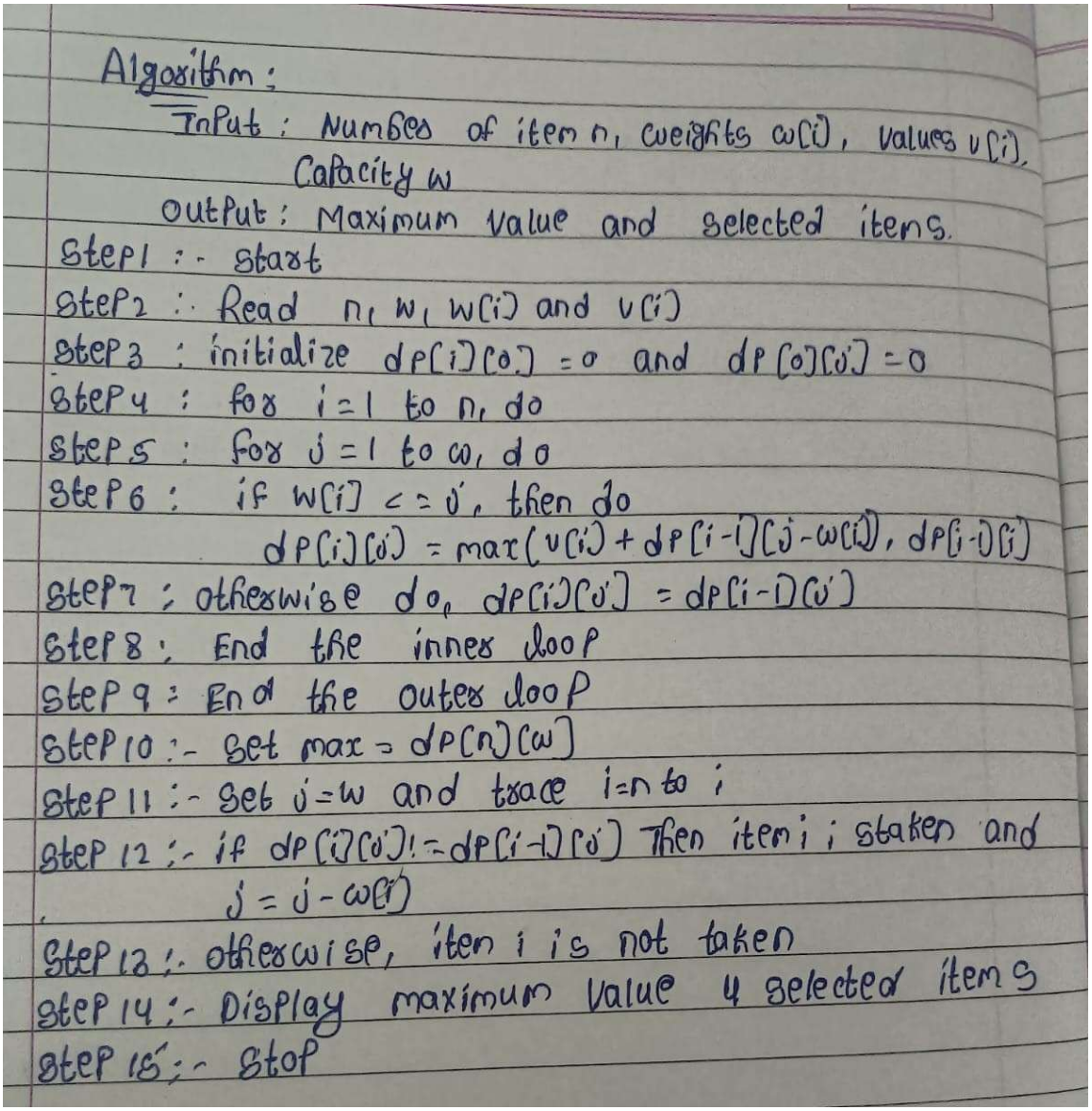
practical-5

Aim: Implementation of a knapsack problem using dynamic programming.

Knapsack:

Algorithm:



Algorithm:
Input : Numbes of item n, weights w[i], values v[i], Capacity W
output : Maximum value and selected items.
Step1 :- start
Step2 : Read n, W, w[i] and v[i]
Step3 : initialize dp[i][0] = 0 and dp[0][j] = 0
Step4 : for i = 1 to n, do
Step5 : for j = 1 to w, do
Step6 : if w[i] <= j, then do
dp[i][j] = max(v[i] + dp[i-1][j-w[i]], dp[i-1][j]
Step7 : otherswise do, dp[i][j] = dp[i-1][j]
Step8 : End the inner loop
Step9 : End the outer loop
Step10 :- Set max = dp[n][w]
Step11 :- Set j = w and trace i = n to i
Step12 :- if dp[i][j] != dp[i-1][j] Then item i staken and j = j - w[i]
Step13 :- otherswise, item i is not taken
Step14 :- Display maximum value & selected items
Step15 :- Stop

Program:

```
#include<iostream>
using namespace std;

class Knapsack {

private:
   int w[10];       // weights
   int v[10];       // values
   int n;           // number of items
   int W;           // capacity
   int dp[10][100]; // dp table

public:

   // ── Get Input ──
   void getInput() {
      cout << "Enter number of items  : ";
      cin  >> n;
      cout << "Enter knapsack capacity : ";
      cin  >> W;
      for(int i = 1; i <= n; i++) {
         cout << "Item " << i << " weight and value : ";
         cin  >> w[i] >> v[i];
      }
   }

   // ── Build dp table ──
   void solve() {

      // base case — row 0 and col 0
      for(int i = 0; i <= n; i++) dp[i][0] = 0;
      for(int j = 0; j <= W; j++) dp[0][j] = 0;


       // fill table
      for(int i = 1; i <= n; i++) {
         for(int j = 1; j <= W; j++) {
```

```
            if(w[i] <= j) {
            // take or not take
            dp[i][j] = max(
               v[i] + dp[i-1][j-w[i]],
               dp[i-1][j]
            );
         } else {
            // cannot take item i
            dp[i][j] = dp[i-1][j];
         }
      }
   }
}

// ── Print dp table ──
void printTable() {
   cout << "\nDP Table:" << endl;
   cout << "     ";
   for(int j = 0; j <= W; j++)
      cout << j << "  ";
   cout << endl;

   for(int i = 0; i <= n; i++) {
      cout << "i=" << i << "  ";
      for(int j = 0; j <= W; j++)
         cout << dp[i][j] << "  ";
      cout << endl;
    }
}

// ── Find selected items ──
void findItems() {
   cout << "\nSelected Items:" << endl;
   int j = W;
   for(int i = n; i >= 1; i--) {
      if(dp[i][j] != dp[i-1][j]) {
         cout << "Item " << i
            << " (w=" << w[i]
               << ", v=" << v[i]
            << ") TAKEN " << endl;
```

```
                j = j - w[i];
            } else {
                cout << "Item " << i
                    << " (w=" << w[i]
                    << ", v=" << v[i]
                    << ") NOT taken " << endl;
            }
        }
    }

    // ── Show result ──
    void showResult() {
        cout << "\nMaximum Value = "
             << dp[n][W] << endl;
    }
};

// ── MAIN ──
int main() {

    Knapsack k;

    k.getInput();
    k.solve();
    k.printTable();
    k.showResult();
    k.findItems();

    return 0;
}
```

Output:

Output

```
Enter number of items   : 4
Enter knapsack capacity : 5
Item 1 weight and value : 6 7
Item 2 weight and value : 8 3
Item 3 weight and value : 9 2
Item 4 weight and value : 7 9

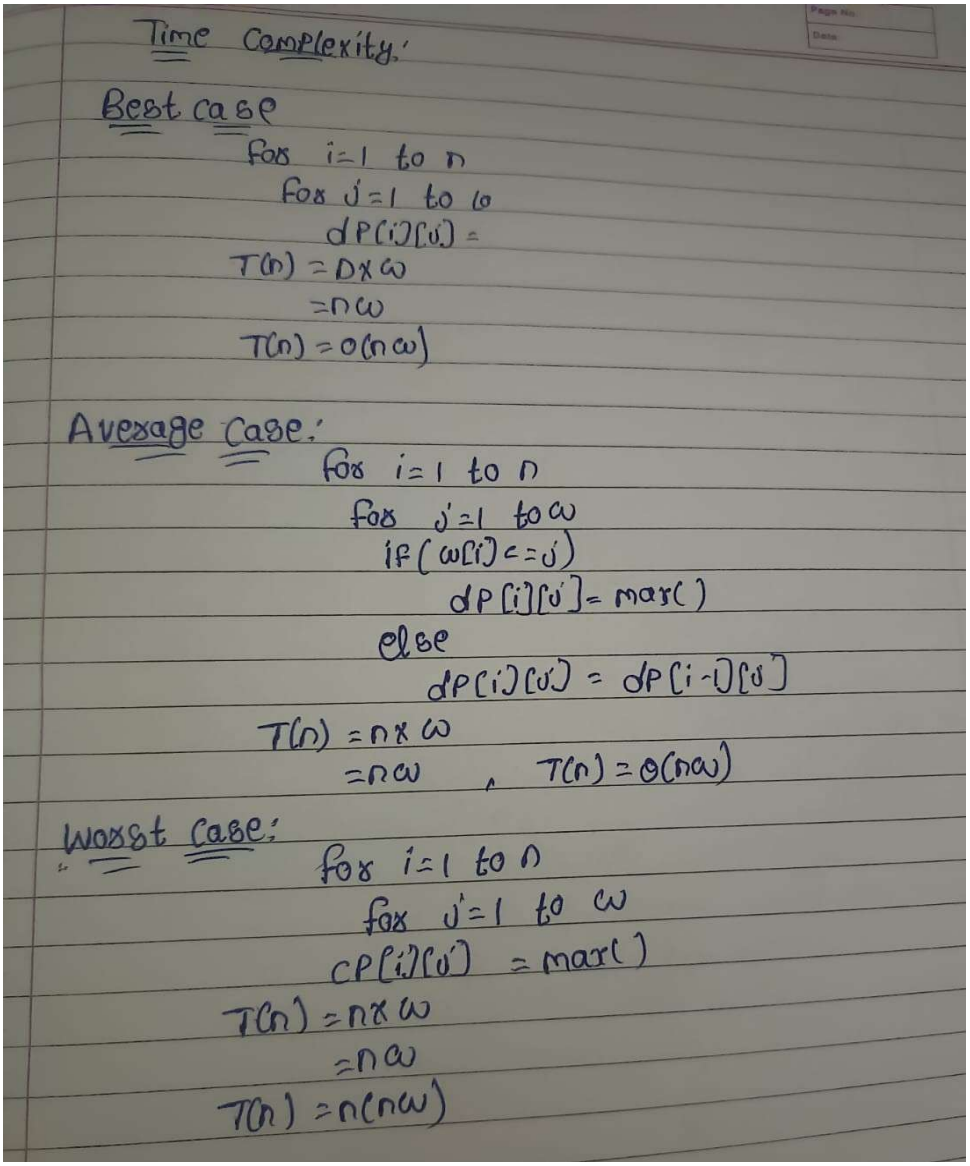
DP Table:
     0  1  2  3  4  5
i=0  0  0  0  0  0  0
i=1  0  0  0  0  0  0
i=2  0  0  0  0  0  0
i=3  0  0  0  0  0  0
i=4  0  0  0  0  0  0

Maximum Value = 0

Selected Items:
Item 4 (w=7, v=9) NOT taken
Item 3 (w=9, v=2) NOT taken
Item 2 (w=8, v=3) NOT taken
Item 1 (w=6, v=7) NOT taken


=== Code Execution Successful ===
```

Time Complexity:



Conclusion:

The implementation and time analysis of 0/1 Knapsack using Dynamic Programming helps us understand its efficiency. The algorithm builds a DP table and considers whether to include or exclude each item to find the maximum value within the given capacity. It has a time complexity of O(nW) in the best, average, and worst cases. Therefore, Dynamic Programming provides an efficient and consistent solution for solving the 0/1 Knapsack problem.